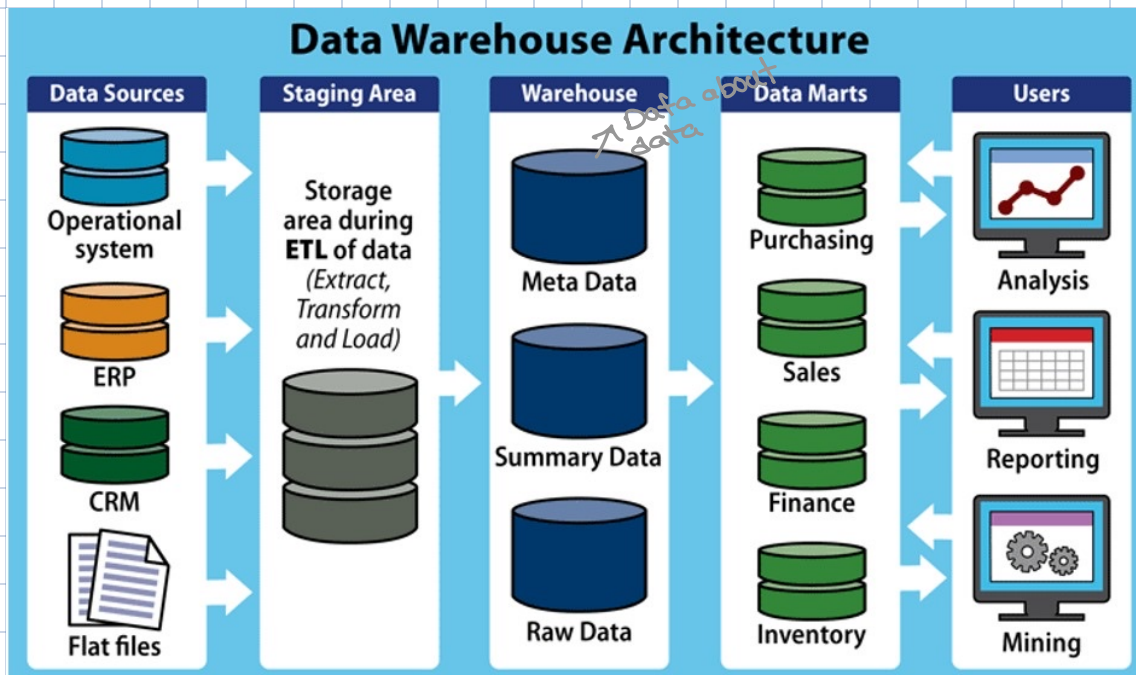


Data Warehouse:

- ↳ Stores large volume of data
- ↳ Passes through a pipeline



• Data Sources:

- Raw data comes from places like sales systems, CRM, finance tools, inventory records, and flat files (e.g., Excel).

• Staging Area:

- Data is cleaned and organized here using ETL (Extract, Transform, Load):
 - **Extract:** Pull data from sources.
 - **Transform:** Fix errors, format, or summarize.
 - **Load:** Send it to the warehouse.

• Warehouse:

- Stores three types of data:
 - Raw Data (untouched).
 - Summary Data (simplified for quick use).
 - Metadata (data about the data, like labels or rules) is managed here too.

• Data Marts:

- Smaller, focused sections of the warehouse for specific tasks:
 - Analysis (find trends).
 - Reporting (create charts/reports).
 - Mining (discover hidden patterns).

• Users:

- Business teams use the data marts to make decisions, like improving sales or managing inventory.

Think of it like a factory: Raw materials (data) are processed (ETL), stored (warehouse), packaged into products (data marts), and delivered to customers (users).

Basic Definition:

1. Database:

↳ A logical collection of related data

2. Data:

↳ Known facts

3. Mini-World:

↳ Some part of the real world about which data is stored in the database.

4. Database Management System:

↳ A software that provides functionality to create & maintain a database.

↳ Retrieval, Modification, Accessing

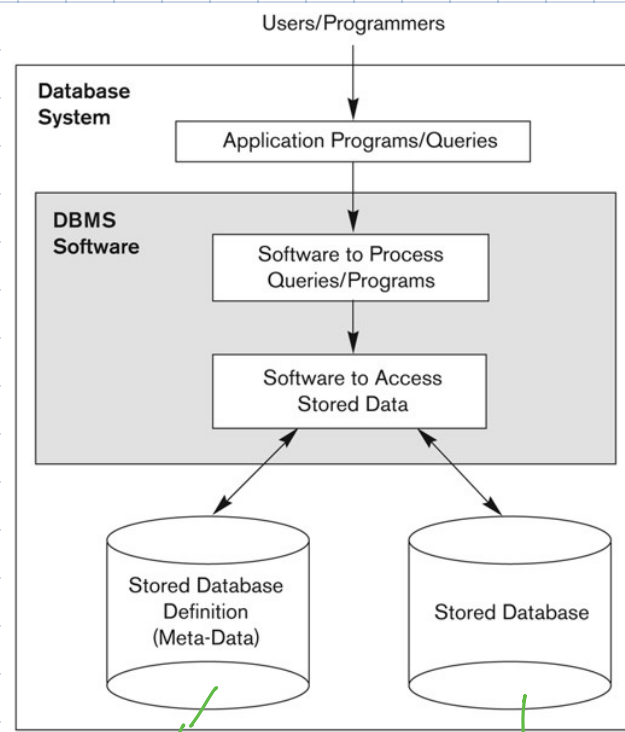
↳ Processing and sharing by a set of concurrent users & application programs.

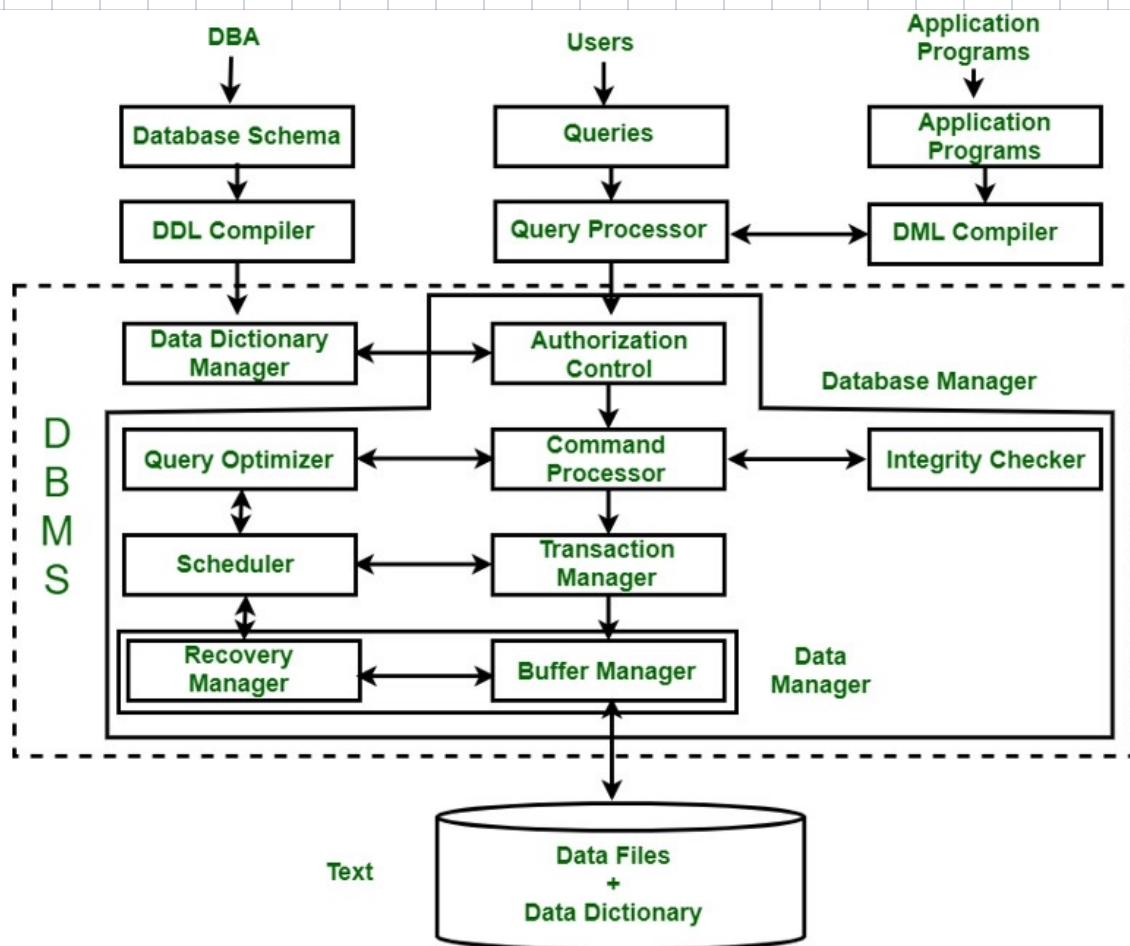
↳ Protection to prevent unauthorized access

↳ Presentation & visualization of data.

5. Database System:

↳ Software + Data + Application (sometimes)





- **DDL Compiler (Sets up the database structure)**

- **Data Dictionary Manager:** Stores metadata (e.g., table names, column types, relationships). Think of it as the "rulebook" for the database.
- **Query Optimizer:** Finds the fastest way to execute queries (like choosing the quickest route on a map).
- **Scheduler:** Manages the order of tasks to avoid conflicts (e.g., two users editing the same data at once).
- **Recovery Manager:** Restores the database after crashes or errors (like a "time machine" for data).
- **Transaction Manager:** Ensures transactions (Command) either fully complete or fail without partial changes.
 - **Example,**
 - If a user tries to delete a product while another user is buying it, the Transaction Manager prevents conflicts.
- **Buffer Manager:** Stores frequently used data in memory for quick access (like a "short-term memory" for the database).
- **Data Files + Data Dictionary:** Where raw data (e.g., customer records) and metadata are physically stored on disk.

- **Users (How people interact with the database)**

- **Queries:** Requests for data (e.g., "Show all orders from June").
- **Query Processor:** Breaks down queries into steps the database can execute.
- **Authorization Control:** Checks if users have permission to perform actions (e.g., "Can John delete this file?").
- **Command Processor:** Executes commands (e.g., updating a password or deleting a record).
- **Transaction Manager:** Same as above—ensures transactions are reliable and consistent.
- **Buffer Manager:** Same as above—manages memory to speed up data access.
- **Data Manager:** Directly reads/writes data to storage (the "bridge" between memory and disk).
- **Application Programs (Software that uses the database)**
 - **Application Programs:** Tools like websites or apps that rely on the database (e.g., a banking app).
 - **DML Compiler:** Translates Data Manipulation Language (e.g., SQL commands like INSERT or UPDATE) into code the database understands.
 - **Database Manager:** Coordinates all activities (the "conductor" of the database orchestra).
 - **Integrity Checker:** Enforces rules to keep data valid (e.g., "A user's age cannot be negative").

Problems with file systems:

↳ Data is

↳ redundant

↳ Unstructured

↳ Requires high maintenance cost

↳ Data dependence

↳ Ensuring Data consistency and controlling access to Data

↳ Difficulties in developing new application

↳ Almost impossible to evolve with new capabilities.

Data base

File System

↳ Coordinates both physical & logical access to data

↳ Coordinates only physical access

↳ Data shared by all programs authorized to have access to it

↳ Data written by one program may not be readable by another

↳ Multiple Users accessing the same system

↳ No two programs can access the same file simultaneously

↳ Flexible access to data (i.e. Queries)

↳ Pre-determined access to Data.

Main characteristics of the Database Approach:

1. Self describing nature of a database system
↳ Stores info about data structures, types and constraints
2. Insulation b/w program & data
3. Data Abstraction
↳ Abstracts away how data is actually stored
4. Supports multiple views of the data
5. Sharing of data & multi-users transaction processing

Database Users:

↳ There are two types:

1. Actors on scene

- a. Database administrators
- b. Database Designers
- c. End Users

- I. Casual → Accesses data occasionally
- II. Naïve → Bank tellers, reservation clerks
- III. Sophisticated → Business analyst, scientists
- IV. Standalone → personal database

When NOT to use DBMS:

- ↳ Unable to handle Data complexity
- ↳ Need special ops not supported